

Secure Storage in Edge-Cloud MPU-based

Bagranov Gleb 560671

As the project for System Security

Table of Contents

1. Executive Summary	3
2. System Architecture	3
3. Security Layers (Defenses)	3
4. Cryptographic Implementation	4
5. Attack Simulations & Testing	5
6. Legend & Terminology	6

1. Executive Summary

This project demonstrates a security-focused IoT simulation establishing a complete "Chain of Trust" between an Edge device (MPU) and a Cloud Server. The architecture is designed to protect data integrity and confidentiality even when communicating across untrusted networks.

2. System Architecture

The system implements four distinct layers of cryptographic defense:

- **Secure Boot:** This layer verifies the integrity of the application code using a SHA-256 hash before any execution begins.
- **Confidentiality:** To prevent unauthorized access to sensitive sensor readings, data-in-transit is protected using AES-256 GCM encryption.
- **Authenticity:** The system utilizes RSA-PSS Digital Signatures to sign outgoing data, ensuring that the Cloud Server can verify the origin and detect any tampering.
- **Anti-Replay:** To prevent attackers from re-sending valid old messages, the system validates UTC timestamps to ensure "freshness".

3. Implementation Details

3.1 Edge MPU Simulator (edge_app.py)

The Edge device follows a rigorous sequence to secure data before transmission:

1. **Capture & Envelope:** It generates a sensor reading (e.g., temperature) and wraps it in a JSON envelope with a current UTC timestamp.
2. **Encryption:** The envelope is encrypted into an `encrypted_blob` using the `SecureMpuStorage` utility.

3. **Digital Signing:** The encrypted_blob is signed with an RSA private key using PSS (Probabilistic Signature Scheme) padding and SHA-256.
4. **Transmission:** The encrypted data is sent via an HTTP POST request with the signature included in the X-Signature header.

3.2 Environment Initialization

The system must be initialized in a specific sequence to establish the cryptographic foundation:

1. **Key Generation:** Run python generate_keys.py to create the necessary RSA and AES keys.
2. **Integrity Fingerprinting:** Run python generate_integrity.py to create the Secure Boot hash for the edge application.
3. **System Launch:** Execute docker-compose up --build to start the containerized environment.

4. Testing & Validation

The project includes specialized scripts to verify that the defenses correctly identify and block various attack vectors:

Script	Security Objective	Test Description
decrypt_check.py	Confidentiality	Verifies that AES ciphertext can be successfully decrypted into readable plaintext by authorized parties.
mitm_attack.py	Integrity & Authenticity	Simulates a Man-in-the-Middle attack by tampering with data; verifies that the cloud detects the change and rejects the packet.

Script	Security Objective	Test Description
hacker_sim.py	Anti-Replay Logic	Attempts to send expired timestamps to confirm the system rejects stale or replayed messages.

5. Attack Simulations & Testing

The system was stressed against three primary attack vectors:

Tampering Attack: Modifying the encrypted bits in transit. Blocked by RSA-PSS signature verification.

Replay Attack: Capturing a valid packet and re-sending it. Blocked by UTC timestamp validation on the Cloud.

Eavesdropping: Attempting to read intercepted data. Blocked by the 256-bit AES key strength.

6. Terminology

Plaintext: The original, readable data (e.g., Temperature: 22°C).

Ciphertext: The scrambled, unreadable version of data after encryption.

SHA-256: A one-way hashing algorithm that creates a unique 256-bit digital fingerprint.

RSA Private Key: A secret key used by the Edge device to "sign" messages.

RSA Public Key: A shared key used by the Cloud to "verify" signatures.

AES-GCM: Advanced Encryption Standard in Galois/Counter Mode; provides both privacy and a tamper-evident seal.

Nonce/IV: A "Number used Once" to ensure that the same data encrypted twice results in different ciphertext.

MITM: Man-in-the-Middle; an attack where the hacker secretly relays and alters communication between two parties.